```
import tkinter as tk
from tkinter import scrolledtext
import nltk
import string

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Download NLTK resources (first time only)
nltk.download('punkt')
nltk.download('stopwords')

# FAQ Dataset
faq_data = {
    "What is your return policy?":
        "You can return products within 30 days of purchase.",

    "How can I track my order?":
        "You can track your order using the tracking link sent to your email.",

    "Do you offer free shipping?":
        "Yes, free shipping is available on orders above ₹500.",

    "How do I contact customer support?":
        "You can contact customer support at support@example.com.",

    "What payment methods are accepted?":
        "We accept credit cards, debit cards, UPI, and net banking."
}

# Text Preprocessing
def preprocess(text):
    text = text.lower()

    tokens = word_tokenize(text)

    tokens = [
        word for word in tokens
        if word not in stopwords.words('english')
        and word not in string.punctuation
    ]

    return " ".join(tokens)

# Prepare FAQ Questions
questions = list(faq_data.keys())
processed_questions = [preprocess(q) for q in questions]

# TF-IDF Vectorization
vectorizer = TfidfVectorizer()
faq_vectors = vectorizer.fit_transform(processed_questions)

# Find Best Matching FAQ
def get_response(user_query):
    processed_query = preprocess(user_query)

    query_vector = vectorizer.transform([processed_query])

    similarities = cosine_similarity(query_vector, faq_vectors)

    best_match_index = similarities.argmax()
    best_score = similarities[0][best_match_index]
```

```
    if best_score < 0.2:
        return "Sorry, I couldn't find a relevant answer."

    matched_question = questions[best_match_index]
    return faq_data[matched_question]

# Send Message
def send_message():
    user_input = entry.get()

    if user_input.strip() == "":
        return

    chat_area.insert(tk.END, "You: " + user_input + "\n")

    response = get_response(user_input)

    chat_area.insert(tk.END, "Bot: " + response + "\n\n")

    entry.delete(0, tk.END)

# GUI
root = tk.Tk()
root.title("FAQ Chatbot")
root.geometry("600x500")

chat_area = scrolledtext.ScrolledText(root, wrap=tk.WORD)
chat_area.pack(padx=10, pady=10, fill=tk.BOTH, expand=True)

entry = tk.Entry(root, width=50)
entry.pack(side=tk.LEFT, padx=10, pady=10, fill=tk.X, expand=True)

send_btn = tk.Button(root, text="Send", command=send_message)
send_btn.pack(side=tk.RIGHT, padx=10)

root.mainloop()
```